KubeBalancer

An Image Classification Pipeline on Kubernetes — Milestone 1: Architecture

Angeudy Brisita Perez

October 5, 2026

Abstract

KubeBalancer is a four-service image classification pipeline deployed on Kubernetes. A user uploads an image to a single public endpoint and receives a predicted label with a confidence score; the original image, the preprocessed image, and the prediction are persisted and remain retrievable by identifier. This document is the Milestone 1 design: it fixes the pod and container decomposition, the Service types and the one external entrypoint, the scaling target, the inter-service API contracts, the persistence layout, and the risk and test plan — before any implementation begins. Every performance figure in this document is a design budget, not a measured result.

Contents

Document Information **1**

1 Overview **2**

2 Use Cases and Workflow **3**

3 Architecture **4**

4 API Contracts **9**

5 Persistence Plan **12**

6 Risk and Test Plan **13**

7 Team Charter **15**

Document Information

Course CSC 478 — [Course Title]
Instructor [Instructor Name]
Student Angeudy Brisita Perez — solo project
Milestone 1 of 3 — Architecture (Design Only)
Submitted October 5, 2026
Repository [repository URL]
Revision 1.0
Project approval Follows the instructor-provided template idea (Image Classification Pipeline on Kubernetes)

This document is a **design artifact**. No application code, container images, or Kubernetes manifests are delivered

at this milestone — by design, since the purpose of Milestone 1 is to settle technical choices before implementation begins. Section references to Milestone 2 and Milestone 3 indicate where each decision will be exercised and verified.

1 Overview

1.1 The problem

Image classification is cheap to prototype and expensive to operate. A model that runs in twenty lines of Python on a laptop still has to answer a harder set of questions before anyone else can use it: what happens when two hundred people upload at once, when a request arrives while the model is still loading, when someone uploads a 40-megapixel photograph or a text file renamed to `.jpg`, or when the process holding every uploaded image is killed? None of those are modelling problems. They are orchestration problems, and they are what this project is about.

1.2 What KubeBalancer is

KubeBalancer is a small, honest, fully-observable image classification service running on Kubernetes. A user sends an image to one public endpoint and gets back a label and a confidence score. Behind that endpoint, four independently deployable services divide the work: a **gateway** that validates and orchestrates, a **preprocessor** that normalizes images into model input, an **inference** service that runs MobileNetV2, and a **storage** service that owns the only persistent volume in the system. Every classification is durable — the original image, the processed image, and the prediction are all retrievable afterwards by identifier.

The design deliberately favours *explicability over scale*. Each service does one thing, exposes one small HTTP contract, and can be tested on its own. The inference tier runs three replicas and carries a metrics sidecar in the same pod, so the behaviour of the component under load is visible rather than inferred. The system is sized to run end-to-end on a single-node Minikube cluster on a laptop, which means every claim in this report is one a reader can reproduce.

1.3 Why it is built this way

The pipeline shape — upload, preprocess, infer, persist, retrieve — is the shape of essentially every production inference service. Keeping it small makes the interesting parts visible. Three decisions carry most of the design's weight, and each is a genuine tradeoff rather than a default:

1. **One orchestrating gateway rather than a chain of services.** It costs an extra network hop and makes the gateway the busiest component, but it keeps preprocessing and inference as pure transforms and gives the system exactly one external surface (§3.2, §3.4).
2. **Synchronous request handling rather than a queue.** A queue would scale better and degrade more gracefully, but it would move the project's difficulty from Kubernetes into distributed messaging, which is not what this project is meant to demonstrate (§3.7).
3. **One service owns all persistent state.** The storage service holds the only volume claim and is pinned to a single replica. This trades horizontal scalability in the storage tier for a system with no distributed-write problem anywhere in it (§5.3).

1.4 Engineering case study framing

This report is written as a case study rather than a feature list, per the project brief. It states the operational problem above; §3 presents the architecture and its central tradeoff; §6 enumerates the risks taken on and how each is contained; and §5.4 and §6.3 set out exactly what evidence will be captured in Milestones 2 and 3 to show deployment, failure recovery, and persistence actually working.

One discipline is worth stating explicitly, because it is easy to violate in a document written before anything runs: **this report claims no measured performance, availability, or scale.** Latency figures appear only as *budgets* — targets

the implementation must meet — and are labelled as such wherever they occur. Measured numbers appear for the first time in the Milestone 3 report.

2 Use Cases and Workflow

Three user stories define the scope of the system. US-1 and US-2 are end-user facing and together describe the full data path; US-3 is operator-facing and is what makes the system a Kubernetes project rather than a web application. Each story carries acceptance criteria that map directly onto rows of the test matrix in §6.3.

2.1 US-1 — Classify an uploaded image

> **As a** user, **I want to** upload an image to a single public endpoint and receive a predicted label with a confidence score, **so that** I can classify images without installing anything.

Acceptance criteria

- `POST /api/v1/images` with a JPEG or PNG under 8 MB returns HTTP 200 with a JSON body containing `imageId`, `label`, `confidence`, and a `topK` list of the three highest-scoring classes.
- The response includes per-stage timings, so the cost of each hop is visible to the caller.
- An upload over the size limit is rejected with HTTP 413 and a structured error body; an upload that is not a decodable image is rejected with HTTP 415. Neither causes a pod restart.
- *Verified by:* T1, T2, T3, T4, T5, T6.

2.2 US-2 — Retrieve a past classification

> **As a** user, **I want to** retrieve a previous prediction and both image variants by their identifier, **so that** results are durable and auditable after the fact.

Acceptance criteria

- `GET /api/v1/results/{imageId}` returns the stored prediction JSON for any previously accepted upload, and HTTP 404 for an unknown identifier.
- `GET /api/v1/images/{imageId}?variant=original` and `?variant=processed` each return the corresponding image bytes with the correct content type.
- Results survive the destruction and recreation of the storage pod: the same identifiers remain retrievable, byte-for-byte, after the pod is deleted and rescheduled.
- *Verified by:* T7, T11.

2.3 US-3 — Scale and survive failure

> **As the** operator, **I want to** scale the inference tier and lose individual pods without dropping user traffic, **so that** the service stays available under load and during failures.

Acceptance criteria

- The inference Deployment scales from 1 to 3 replicas with all replicas reaching `Ready`, and requests are observably distributed across them via the per-pod metrics sidecar.
- Manually deleting an inference pod results in automatic recreation by the Deployment controller, with no failed user requests while the remaining replicas serve traffic.
- With the inference tier scaled to zero, the gateway returns a well-formed HTTP 503 rather than hanging or crashing, and recovers automatically when replicas return.
- A full teardown and reprovision (`kubectl delete -f . && kubectl apply -f .`) brings the whole stack back to `Ready` without manual intervention.
- *Verified by:* T8, T9, T10, T12, T13.

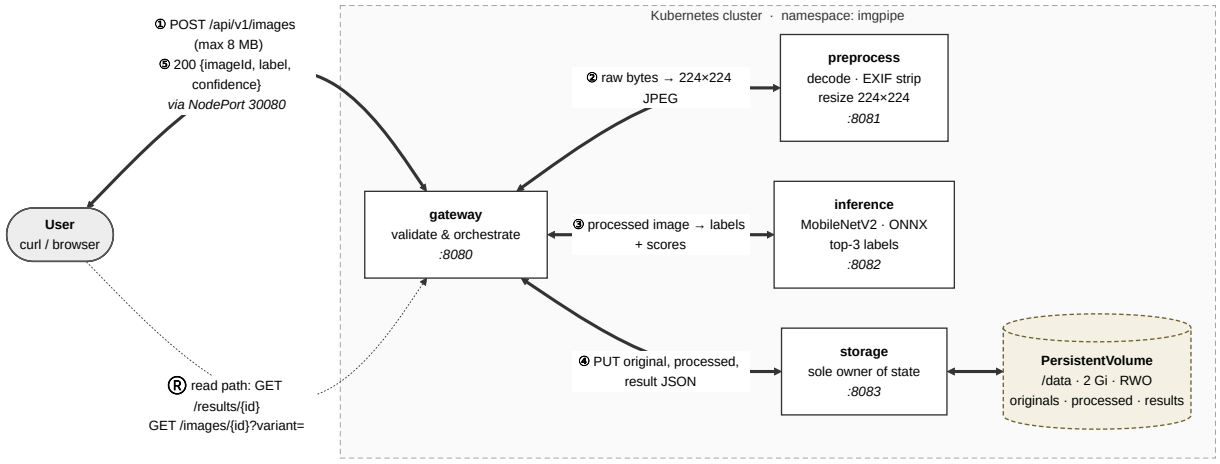


Figure 1: KubeBalancer data flow. The write path (solid, numbered) runs upload → preprocess → inference → persist → response. The read path (dashed) serves retrieval from the persistent volume. Only the gateway is reachable from outside the cluster.

2.4 Data flow

Figure 1 shows the request path for both stories. The write path (US-1) is a single synchronous request that fans through three internal services before responding. The read path (US-2) touches only the gateway and the storage service. The storage service is the sole holder of persistent state, and the sole component that is never exposed outside the cluster.

3 Architecture

3.1 Design goals and constraints

Four constraints shape the architecture — three from the project brief, one from the deployment target.

1. **Exactly one external entrypoint.** All internal traffic uses `ClusterIP`; precisely one Service is reachable from outside the cluster.
2. **At least one multi-container pod**, with a defensible reason for co-location rather than separate Deployments.
3. **One credible scaling target**, run at three or more replicas and shown to self-heal.
4. **Single-node Minikube.** No cloud load-balancer controller and no multi-node scheduling, so Service types and volume access modes are chosen for what exists on one node rather than for a managed cluster.

Two principles follow. **Each service is one transform behind one HTTP contract**, so it can be exercised with a single `curl` command while the rest of the system is down. **Persistent state has exactly one owner**, which keeps every scaling and failure question in the system a stateless one.

3.2 Component decomposition

The pipeline is four Deployments. Table 1 lists them with their replica counts at Milestone 2 and their responsibilities.

Table 1: Component list. The inference pod is the only multi-container pod. Replica counts are the Milestone 2 targets.

Component	Containers	Replicas	Port(s)	Responsibility
`gateway`	1	1	8080	Sole external entrypoint. Validates the upload, orchestrates the three internal hops, returns the prediction, serves retrieval.
`preprocess`	1	2	8081	Decodes the image, strips EXIF, resizes to 224×224, normalizes to model input. Pillow.

Component	Containers	Replicas	Port(s)	Responsibility
`inference`	**2**	**3**	8082, 9100	MobileNetV2 forward pass via ONNX Runtime, returns top-*k* labels. Carries the metrics sidecar.
`storage`	1	1 (pinned)	8083	Owns the only PersistentVolumeClaim. Persists original images, processed images, and prediction JSON.

Why a separate gateway. The obvious alternative is a chain: expose the preprocessor, have it call inference, have inference call storage. That removes a pod and a network hop. It was rejected for two reasons. First, it gives the preprocessor two unrelated jobs — image transformation and request orchestration — which makes it the hardest service to test and the one most likely to change for reasons unrelated to preprocessing. Second, retrieval has nowhere sensible to live: the storage service would have to be exposed externally, which breaks constraint 1 and puts the component holding all persistent state directly on the public surface. A dedicated gateway keeps preprocessing and inference as pure functions of their input and leaves exactly one component with any knowledge of the pipeline's shape.

Why preprocessing is a service rather than library code in the gateway. It is the stage most likely to change — input size, normalization, and model family all land here — and it is CPU-bound where the gateway is I/O-bound. Separating them lets preprocessing be scaled or replaced without redeploying the public endpoint. It runs two replicas so the gateway always has a second instance to fail over to.

3.3 The multi-container pod: inference and its metrics sidecar

The `inference` pod runs two containers (Figure 2):

- **`inference`** — the FastAPI application. On each request it appends one JSON line to `requests.jsonl` on a shared volume mounted at `/var/log/app/`, recording the timestamp, request ID, latency in milliseconds, model version, and outcome.
- **`metrics-exporter`** — a sidecar that tails that same file, maintains a rolling aggregate (request count, error count, p50 and p95 latency), and serves it in Prometheus text format on port 9100.

The two containers share an `emptyDir` volume mounted at `/var/log/app/` and communicate over `localhost`. This is the canonical justification for co-location rather than separation: the sidecar needs filesystem access to a file the application writes and must observe *this specific replica*, not the tier in aggregate. Two separate Deployments physically cannot share an `emptyDir`, and routing through a Service would destroy the per-replica attribution that makes the metrics useful.

Placing the sidecar on `inference` rather than on the gateway is deliberate. `inference` is the component being scaled, so every new replica brings its own exporter. That turns the Milestone 2 scaling demonstration from an assertion into an observation: with three replicas, three distinct `:9100` endpoints report non-zero and roughly balanced request counts, which is direct evidence that the Service is load-balancing rather than pinning traffic to one pod. An access-logging sidecar on the single-replica gateway was considered and rejected — it would have produced a weaker demonstration of why pods group containers at all.

The sidecar is also where the design accepts a known limitation: it aggregates in memory, so its counters reset when the pod restarts, and there is no Prometheus server scraping it in this project. Metrics are read directly with `kubectl port-forward` and `curl`. Adding a real monitoring stack is listed as a next step rather than claimed as a feature.

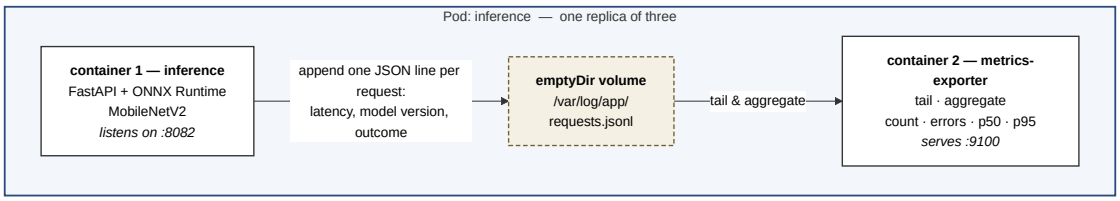


Figure 2: The multi-container inference pod. The two containers share an `emptyDir` volume and the pod's network namespace, which is precisely why they belong in one pod rather than two Deployments. Each of the three replicas carries its own exporter, so per-pod request counts are directly observable.

3.4 Service topology and external exposure

Table 2: Service choices. Only `gateway-svc` is reachable from outside the cluster.

Service	Type	Port mapping	Rationale
`gateway-svc`	**NodePort**	30080 → 8080	The single external entrypoint. Minikube has no cloud load-balancer controller, so a `LoadBalancer` Service would sit in `Pending` indefinitely; NodePort is the honest choice for this target, and `minikube service gateway-svc --url` yields a stable address for testing.
`preprocess-svc`	ClusterIP	8081 → 8081	Internal only. Also load-balances the two preprocessing replicas.
`inference-svc`	ClusterIP	8082 → 8082, 9100 → 9100	Internal only. Distributes requests across the three inference replicas; the metrics port is exposed on the Service for convenience but is reached in practice per-pod.
`storage-svc`	ClusterIP	8083 → 8083	Internal only, and reachable from the gateway alone. The component holding all persistent state is never on the public surface.

Services are addressed by DNS name, not IP — `http://preprocess-svc.imgpipe.svc.cluster.local:8081` — so no manifest contains a hard-coded pod address and rescheduling requires no configuration change. Figure 3 shows the full topology, including which objects sit inside the cluster boundary.

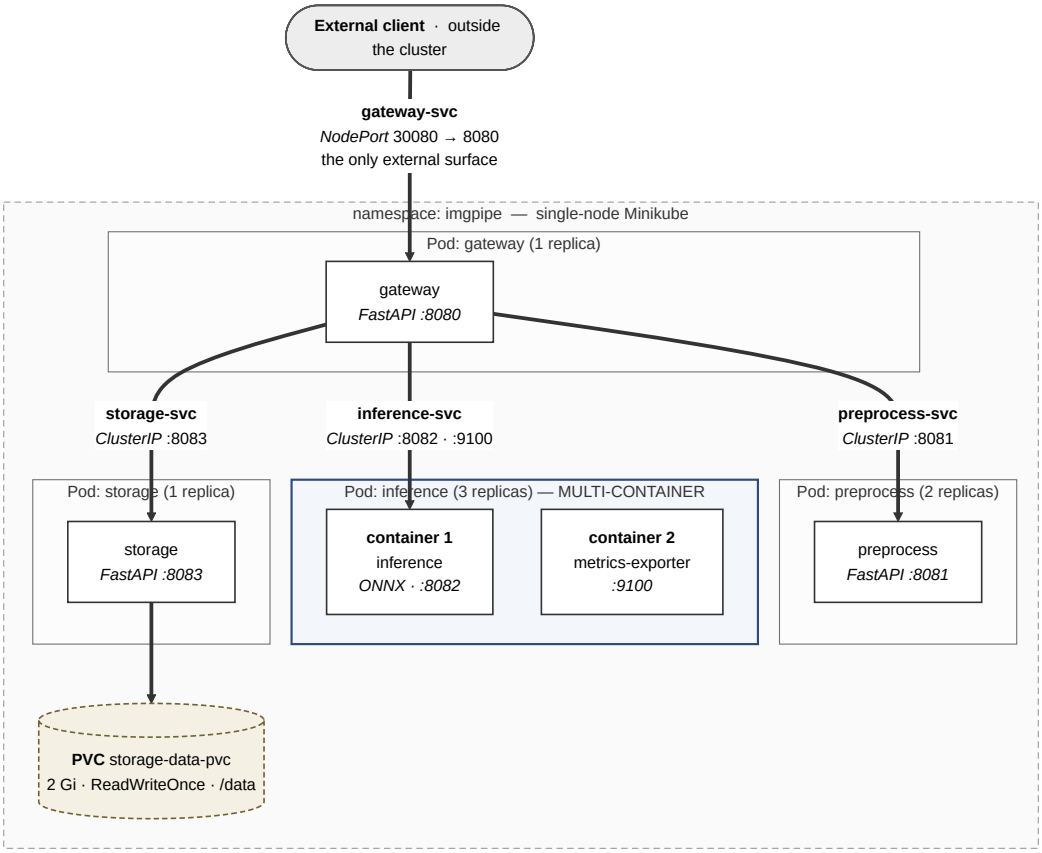


Figure 3: Kubernetes topology. Boxes are containers; outlined regions are pods with their replica counts. Services appear as the labelled arrows between pods — only `gateway-svc` crosses the cluster edge. Full Service details are in Table 2.

3.5 Request lifecycle and timeout budget

A classification request proceeds through four stages. The gateway generates an `X-Request-Id` on arrival and propagates it to every downstream call, so a single request can be traced across all four services' logs.

1. **Validate** — content type against an allowlist, and size against an 8 MB limit, both before the body is read into memory.
2. **Preprocess** — `POST /preprocess` with the raw bytes; receives a 224×224 JPEG back.
3. **Infer** — `POST /predict` with the processed image; receives top-*k* labels and scores.
4. **Persist** — three calls to the storage service: the original, the processed image, and the prediction JSON. Then respond to the client.

Each hop has an explicit timeout, and the sum of the internal budgets is strictly less than the client-facing budget so that a slow dependency produces a clean error rather than a connection reset:

Table 3: Timeout budget. These are *design targets*, not measured latencies. No figure in this table has been benchmarked.

Stage	Timeout budget	Retry policy
Client ↔ gateway	30 s	None — the client owns retries.
Gateway → preprocess	5 s	One retry, connection errors only.
Gateway → inference	15 s	**No retry.** A 5xx from inference is returned as 502; retrying a request that may have been expensive to fail is how a slow tier becomes a dead one.
Gateway → storage	5 s	One retry, connection errors only.

Retrying only on connection errors — never on a 5xx — is a deliberate choice. A connection error means the request never arrived and is safe to repeat; a 5xx means it arrived and failed, and repeating it multiplies load on a component that is already struggling.

3.6 Scaling target

`inference` is the scaling target, running three replicas at Milestone 2. It is the right choice on three grounds: it is stateless, so replicas are interchangeable; it is CPU-bound on the ONNX forward pass, which is the single most expensive operation in the request path; and it therefore holds the largest share of the latency budget (15 s of 30 s), making it the component that saturates first under concurrent load.

The other three tiers are deliberately not scaled. `gateway` stays at one replica because it is I/O-bound and not the bottleneck; scaling it would add capacity where none is needed. `preprocess` runs two replicas for availability rather than throughput. `storage` is **pinned to one replica** by the ReadWriteOnce access mode of its volume, with `strategy: Recreate` rather than `RollingUpdate` — a rolling update would attempt to schedule a second pod against a volume that can only be attached once, and would stall on multi-attach. This is a real constraint of the design, not an oversight, and §5.3 sets out what it costs and what would relieve it.

3.7 The central tradeoff: synchronous orchestration

The most consequential decision in this design is that classification is **synchronous**. The gateway holds the client connection open for the entire pipeline and returns the prediction in the response to the upload.

The alternative is a queue: accept the upload, return `202 Accepted` with a job identifier, have worker pods consume from a broker, and let the client poll. That design is strictly better under load. It absorbs traffic spikes in the queue instead of in timeouts, it lets the inference tier be scaled by queue depth, and it survives a total inference outage without failing user requests.

It was rejected because it would relocate the project's difficulty. A queue introduces a broker pod, at-least-once delivery, idempotency on retried jobs, job-state storage, and client polling logic — a substantial body of distributed-systems work that would consume the implementation time this project is meant to spend on Kubernetes orchestration itself. The synchronous design makes the failure modes visible and bounded instead: the timeout budget in Table 3 is the entire back-pressure mechanism, and when inference is unavailable the gateway fails fast with a 503.

The honest cost is stated plainly: **KubeBalancer has no back-pressure beyond its timeouts.** Under sustained concurrent load past the inference tier's capacity, requests will time out rather than queue. Moving to a queue-backed design is the first item in the Milestone 3 next-steps section.

3.8 Resilience and resource policy

- **Probes.** Every service exposes `/healthz` for liveness and `/readyz` for readiness as separate endpoints. The distinction matters most on `inference`, where `/readyz` returns 200 only once the ONNX model is loaded. Without it, the Service routes traffic to a pod that is running but cannot yet answer, producing spurious 503s on every scale-up and restart.
- **Resources.** Inference containers request 256 Mi / 250 m, limited to 768 Mi / 1000 m, with `OMP_NUM_THREADS=1`: ONNX Runtime otherwise sizes its thread pool to the host core count, and three full-width pools on one node contend badly. Other services request 64 Mi / 50 m, limited to 256 Mi / 500 m.
- **Images.** Explicit version tags (`kubebalancer/inference:0.1.0`), never `latest`; built locally, loaded with `minikube image load`, and `imagePullPolicy: IfNotPresent` so the cluster never reaches for a registry that has no copy. MobileNetV2 weights are baked into the image rather than fetched by an init container, removing network dependency from pod startup.

3.9 Namespace, labels, and repository layout

All objects live in a dedicated `imgpipe` namespace, which makes teardown unambiguous. Every object carries the standard recommended labels (`app.kubernetes.io/name`, `/component`, `/part-of: kubebalancer`, `/version`), so `kubectl` selectors and Service definitions stay consistent.

Manifests are numerically prefixed so `kubectl apply -f .` applies them in dependency order — namespace, configuration, volume claims, Deployments, Services. This is what makes the reproducibility requirement (`kubectl delete -f . && kubectl apply -f .`) work without manual sequencing:

```
deployments/   00-namespace.yaml  10-gateway.yaml  11-preprocess.yaml
               12-inference.yaml (2 containers)    13-storage.yaml
services/      20-gateway-svc.yaml (NodePort)      21-preprocess-svc.yaml
               22-inference-svc.yaml               23-storage-svc.yaml
storage/       05-storage-pvc.yaml                 (05-storage-pv.yaml, static fallback)
services-src/  gateway/  preprocess/  inference/  metrics-sidecar/  storage/
```

These directories are created in Milestone 2; the layout is fixed here so the report, the repository, and the test scripts agree from the start.

4 API Contracts

All four services speak JSON over HTTP and are implemented with FastAPI on Python 3.11, so each publishes a machine-readable OpenAPI document at `/openapi.json` and interactive documentation at `/docs`. The contracts below are the authoritative definitions for Milestone 2.

Two conventions apply to every endpoint on every service:

- **Request correlation.** The gateway generates an `X-Request-Id` for each inbound request (or honours a client-supplied one) and forwards it on every internal call. All four services include it in their log lines and in error responses.
- **Uniform error envelope.** Every non-2xx response has the same shape, so the gateway can translate a downstream failure without inventing a format:

  ```
  { "error": { "code": "PAYLOAD_TOO_LARGE",
               "message": "Upload exceeds the 8388608 byte limit" },
    "requestId": "7f3c1e9a" }
  ```

  Note on the figures below. Latency values in these examples are *illustrative placeholders* showing the response shape. No latency in this document has been measured; see Table 3.

4.1 Gateway (external — the only public API)

`POST /api/v1/images` — classify an image

Accepts `multipart/form-data` with a single file field. Maximum 8 MB; `image/jpeg` and `image/png` only.

```
curl -s -X POST "http://$(minikube ip):30080/api/v1/images" \
     -F "file=@samples/retriever.jpg;type=image/jpeg"
```

```
{ "imageId": "img_01JAR7K2M9QX4T8B",
  "label": "golden retriever",
  "confidence": 0.8734,
```

```
  "topK": [ { "label": "golden retriever",   "score": 0.8734 },
            { "label": "Labrador retriever", "score": 0.0612 },
            { "label": "kuvasz",             "score": 0.0188 } ],
  "modelVersion": "mobilenetv2-1.0-onnx",
  "mock": false,
  "timings": { "preprocessMs": 38, "inferenceMs": 121,
               "storageMs": 44, "totalMs": 207 },
  "requestId": "7f3c1e9a" }
```

`GET /api/v1/results/{imageId}` — retrieve a stored prediction

Returns the persisted prediction record, which is the response body above plus a `storedAt` timestamp and the storage paths of both image variants. Returns 404 for an unknown identifier.

`GET /api/v1/images/{imageId}?variant=original|processed` — retrieve image bytes

Returns the raw image with the appropriate `Content-Type`. `variant` defaults to `original`. Returns 404 if the identifier or the requested variant does not exist.

`GET /api/v1/results?limit=20` — list recent classifications

Returns `{ "items": [ ... ], "count": 20 }`, newest first, backed by the append-only index described in §5.2. Supports `limit` between 1 and 100, defaulting to 20.

Gateway status codes

Table 4: Gateway response codes.

Code	`error.code`	Condition
200	—	Classification or retrieval succeeded.
400	`EMPTY_UPLOAD`	Zero-byte file, or the `file` field is absent.
404	`NOT_FOUND`	Unknown `imageId`, or variant not stored.
413	`PAYLOAD_TOO_LARGE`	Upload exceeds 8 MB. Rejected before the body is buffered.
415	`UNSUPPORTED_MEDIA_TYPE`	Content type not in the allowlist, or bytes are not a decodable image.
502	`UPSTREAM_FAILED`	A downstream service returned 5xx. Not retried (§3.5).
503	`UPSTREAM_UNAVAILABLE`	No healthy replica of a required service.
504	`UPSTREAM_TIMEOUT`	A downstream hop exceeded its budget from Table 3.

4.2 Preprocess (internal)

`POST /preprocess`

Takes raw image bytes — not multipart — with the content type in the header. Decodes, strips EXIF metadata, converts to RGB, resizes to 224×224, and re-encodes as JPEG at quality 90.

```
POST /preprocess HTTP/1.1
Content-Type: image/jpeg
X-Request-Id: 7f3c1e9a

<raw image bytes>
```

```
{ "width": 224, "height": 224, "format": "jpeg", "colorspace": "RGB",
  "bytes": 18442, "sourceWidth": 1600, "sourceHeight": 1200,
  "data": "/9j/4AAQSkZJRgABAQEAYABgAAD...<base64, truncated>" }
```

Returns 415 with `UNSUPPORTED_MEDIA_TYPE` if the bytes do not decode as an image, and 413 if the decoded pixel count exceeds the decompression-bomb guard (§6.1, R6).

4.3 Inference (internal)

`POST /predict`

```
{ "data": "/9j/4AAQSkZJRgABAQEAYABgAAD...<base64 224x224 JPEG>", "topK": 3 }

{ "predictions": [ { "label": "golden retriever",   "score": 0.8734 },
                   { "label": "Labrador retriever", "score": 0.0612 },
                   { "label": "kuvasz",             "score": 0.0188 } ],
  "modelVersion": "mobilenetv2-1.0-onnx",
  "mock": false,
  "latencyMs": 121 }
```

`topK` defaults to 3 and is capped at 10. The `mock` field reports whether the response came from the real MobileNetV2 graph or from the deterministic mock classifier — when `MOCK_MODE=1`, the service derives a stable label from a hash of the input bytes, so tests are repeatable and need no model loaded. The field is part of the contract rather than hidden configuration, so no result can be mistaken for a real prediction.

`GET /metrics` — served by the sidecar on port 9100

Prometheus text format, per replica:

```
kubebalancer_inference_requests_total 143
kubebalancer_inference_errors_total 2
kubebalancer_inference_latency_ms{quantile="0.5"} 118
kubebalancer_inference_latency_ms{quantile="0.95"} 186
```

4.4 Storage (internal)

Table 5: Storage service endpoints. `{variant}` is `original` or `processed`.

Method and path	Request body	Success	Purpose
`PUT /objects/{imageId}/{variant}`	raw bytes	201	Store an image variant. Returns its path and byte count.
`GET /objects/{imageId}/{variant}`	—	200	Retrieve image bytes.
`PUT /results/{imageId}`	JSON record	201	Store the prediction record and append to the index.
`GET /results/{imageId}`	—	200	Retrieve one prediction record.
`GET /results?limit=n`	—	200	List recent records, newest first.

Writes are idempotent on `imageId`: re-`PUT`ting the same identifier overwrites in place and does not duplicate the index entry.

4.5 Health and readiness (all services)

Table 6: Probe endpoints.

Endpoint	Probe	Returns 200 when
`GET /healthz`	liveness	The process is running and the event loop is responsive. Never checks dependencies — a liveness probe that fails on a dependency outage causes a restart storm.
`GET /readyz`	readiness	The service can serve traffic. On `inference`, only after the ONNX model is loaded; on `storage`, only after `/data` is mounted and writable.

5 Persistence Plan

Milestone 2 runs the pipeline with ephemeral storage; Milestone 3 adds the persistent volume described here. The design is fixed now because the storage service's API and directory layout determine what the other services send it.

5.1 What needs to be stored

Each accepted upload produces three durable artifacts:

Table 7: Persisted artifacts per classification. Sizes are design expectations for the sample set, not measurements.

Artifact	Typical size	Why it is kept
Original image	0.1–8 MB	Audit trail. Lets a prediction be re-run against a newer model without asking the user to re-upload.
Processed image (224×224 JPEG)	~20 KB	Shows exactly what the model saw. Makes a surprising prediction diagnosable.
Prediction record (JSON)	~1 KB	The result itself: labels, scores, model version, timings, timestamp.

Nothing else is persisted. There is no user database, no session state, and no request log on disk — logs go to stdout and are read with `kubectl logs`, which is the Kubernetes-native path and keeps the volume holding only user data.

5.2 Volume design and directory layout

A single `PersistentVolumeClaim` named `storage-data-pvc` requests **2 Gi** with access mode **ReadWriteOnce**, using Minikube's default `standard` StorageClass, which provisions dynamically through the `hostPath` provisioner on the node. It is mounted at `/data` in the storage pod and nowhere else. 2 Gi is sized for roughly two thousand sample images — generous for a demonstration and small enough to provision instantly.

```
/data/
├── originals/<yyyy>/<mm>/<dd>/<imageId>.<ext>     original upload, as received
├── processed/<yyyy>/<mm>/<dd>/<imageId>.jpg       224x224 model input
├── results/<imageId>.json                         prediction record
└── index/manifest.jsonl                           append-only, one line per classification
```

Images are sharded by ingest date so no single directory accumulates unbounded entries, while prediction records are flat because they are only ever looked up by exact identifier. The `manifest.jsonl` index exists so `GET /results?limit=n` does not have to scan a directory: it is append-only, so a write is a single `O(1)` operation with no read-modify-write cycle to corrupt under concurrent access, and the listing endpoint reads it from the tail.

If dynamic provisioning misbehaves on the target cluster, a statically-provisioned `hostPath` PersistentVolume is kept as a documented fallback (`storage/05-storage-pv.yaml`), bound explicitly by the claim. The reclaim policy is `Retain` in both cases, so deleting the claim does not immediately destroy the demonstration data.

5.3 The single-writer constraint

ReadWriteOnce means the volume can be mounted for writing by one node at a time, and in practice by one pod. This has three consequences that are consequences of the design, not accidents of it:

1. The storage Deployment is **pinned to one replica**.
2. Its update strategy is **`Recreate`, not `RollingUpdate`** — a rolling update would try to start a second pod while the first still holds the volume, and the new pod would stay `Pending` on a multi-attach error while the old one was never terminated.
3. There is therefore a **brief write outage during any storage redeploy**, bounded by pod startup time. Uploads during that window fail with 503 from the gateway rather than being queued.

This is the cost of having exactly one owner of persistent state, and it buys a system with no distributed-write problem: no two processes ever contend for the same file, so no locking, leader election, or conflict resolution appears anywhere in the design. The production path away from it is an object store (S3 or MinIO) behind the same storage API, which would make the storage service stateless and horizontally scalable without changing a single caller. That is recorded as a next step, not implemented here.

5.4 Verification procedure (executed in Milestone 3)

Persistence is only demonstrated if data survives the destruction of the process that wrote it. The procedure is:

1. Upload three distinct images; record the three returned `imageId` values.
2. Confirm all three are retrievable via `GET /api/v1/results/{imageId}`, and capture the output.
3. Delete the storage pod outright: `kubectl delete pod -l app.kubernetes.io/component=storage -n imgpipe`.
4. Watch the Deployment controller recreate it: `kubectl get pods -n imgpipe -w` until `Ready`.
5. Re-request all three identifiers and the processed image variants. Byte-for-byte identical results constitute the evidence.
6. Repeat with `kubectl rollout restart deployment/storage -n imgpipe` to confirm the `Recreate` strategy behaves as designed rather than stalling.
7. Finally, confirm that `kubectl delete -f . && kubectl apply -f .` reprovisions the full stack, and document explicitly whether data survives it — with `Retain` the volume persists, and that distinction between deleting a pod and deleting a claim is itself part of the result.

Each step's command and output are captured for the Milestone 3 report. No persistence claim is made that is not backed by a captured command.

6 Risk and Test Plan

6.1 Risk register

Eight risks are tracked. Each has a mitigation that is already reflected in the architecture rather than deferred, and each maps to the test that would detect it if the mitigation failed.

Table 8: Risk register. L = likelihood, I = impact, on a Low/Medium/High scale.

#	Risk	L	I	Mitigation (already in the design)	Detected by
R1	**Model size and cold start.** A large runtime image slows scheduling and risks `ImagePullBackOff` or `ErrImageNeverPull` on a local cluster.	M	M	ONNX Runtime with MobileNetV2 (~14 MB weights) on `python:3.11-slim`; weights baked into the image, not downloaded at startup; `minikube image load` with `imagePullPolicy: IfNotPresent`.	T12
R2	**Timeout cascade.** Three chained hops let one slow dependency hang the client connection.	M	H	Per-hop budgets summing to less than the client budget (Table 3); retry on connection errors only, never on 5xx.	T8

#	Risk	L	I	Mitigation (already in the design)	Detected by
R3	**OOMKilled under replication.** Three inference replicas on one node, each sizing an ONNX thread pool to the host core count, exhaust node memory or thrash the CPU.	M	H	Explicit requests and limits (256 Mi/250 m → 768 Mi/1000 m) plus `OMP_NUM_THREADS=1` per replica.	T10, T13
R4	**ReadWriteOnce volume versus scaling.** Any attempt to run two storage pods deadlocks on multi-attach.	L	H	Storage pinned to one replica with `strategy: Recreate`; constraint documented in §5.3 as a design cost.	T11
R5	**Traffic before the model is loaded.** A pod that is `Running` but not yet able to infer receives requests and returns 503s on every scale-up.	H	M	Separate `/healthz` and `/readyz`; readiness gated on completed model load.	T9, T10
R6	**Decompression bombs and oversized uploads.** A small file can decode to gigabytes of pixels and exhaust the preprocessor.	M	H	8 MB limit enforced at the gateway *before* the body is buffered; MIME allowlist; `Pillow.MAX_IMAGE_PIXELS` guard in the preprocessor.	T2, T4, T5
R7	**Grading reproducibility.** The brief requires `kubectl delete -f . && kubectl apply -f .` to cleanly reprovision; dependency ordering or a stale NodePort binding can break it.	M	H	Numerically prefixed manifests applied in dependency order; dedicated `imgpipe` namespace; fixed NodePort 30080 checked for conflicts.	T12
R8	**Solo schedule risk.** A single developer blocked on the ML path stops all progress on orchestration, which is what the project is actually graded on.	M	M	`MOCK_MODE` deterministic classifier means the full pipeline can be built, deployed, and tested with no model present.	T1

The two risks taken most seriously are R2 and R5, because both produce *intermittent* failures that are easy to misread as flakiness. R5 in particular is the failure most likely to appear during the Milestone 3 demonstration, when pods are being deliberately deleted on camera.

6.2 Test approach

Testing is layered, and the layers are kept separate so a failure localizes immediately:

- **Unit** — pure transform logic per service: resize correctness, EXIF stripping, label decoding, path construction. Runs with `pytest`, no network, no cluster.
- **Contract** — each service started alone in a container and exercised with `curl` against the contracts in §4. This is where a mismatch between two services is caught, before they are wired together.
- **Integration** — the full pipeline on the cluster through the NodePort endpoint, driven by a shell script committed to the repository so it is reproducible by a grader rather than described.
- **Operational** — scaling, pod deletion, and persistence survival, verified with `kubectl` and captured as command output.

Throughout Milestone 2 the inference service runs with `MOCK_MODE=1` so that orchestration failures are never confused with model failures. The real MobileNetV2 graph is enabled and the suite re-run before the Milestone 3 demonstration.

6.3 Test matrix

Table 9: Test matrix. M2 tests run at Milestone 2; M3 tests require the persistent volume.

ID	Case	Input / action	Expected result	When
T1	Happy path	500 KB JPEG of a dog	200; plausible label with confidence; three artifacts written	M2
T2	Oversized upload	12 MB JPEG	413 `PAYLOAD_TOO_LARGE`; no pod restart; memory flat	M2
T3	At-limit upload	7.5 MB JPEG	200 within the 30 s client budget	M2

ID	Case	Input / action	Expected result	When
T4	Invalid image	`.txt` file renamed `.jpg`	415 `UNSUPPORTED_MEDIA_TYPE`; structured error; no crash	M2
T5	Unsupported format	GIF, then SVG	415 for both; allowlist holds	M2
T6	Empty upload	Zero-byte file	400 `EMPTY_UPLOAD`	M2
T7	Unknown identifier	`GET /results/img_doesnotexist`	404 `NOT_FOUND`	M2
T8	Dependency unavailable	`kubectl scale --replicas=0 deploy/inference`	503 `UPSTREAM_UNAVAILABLE` promptly, not a hang; full recovery when scaled back	M2
T9	Self-healing	`kubectl delete pod <one inference pod>`	Deployment recreates it; concurrent requests keep succeeding on the surviving replicas	M2
T10	Scaling	`kubectl scale --replicas=3 deploy/inference`	All three `Ready`; all three sidecar `:9100` endpoints report non-zero, roughly balanced counts	M2
T11	Persistence survival	Upload 3, delete storage pod, re-request	Identical bytes and JSON after recreation (§5.4)	M3
T12	Clean reprovision	`kubectl delete -f . && kubectl apply -f .`	Every pod `Ready`, endpoint serving, no manual steps	M2
T13	Concurrency	10 parallel uploads	All return 200; p95 recorded as the first *measured* latency figure in the project	M2
T14	Probe correctness	Restart inference, poll `/readyz` during startup	`/readyz` returns non-200 until the model is loaded; no request is routed to a not-ready pod	M2

T13 deserves a note: it is the first point at which this project produces a measured performance number. Until it runs, every latency figure in this report is a budget (Table 3), and the Milestone 3 report will present measured values beside those budgets so the gap is visible either way.

7 Team Charter

This is a **solo project**, so the charter the brief asks for is adapted rather than omitted: in place of interpersonal agreements it records role separation, a self-imposed cadence, and a written Definition of Done. The separation of roles is genuinely useful even with one person, because the three roles fail in different ways and compete for the same hours.

7.1 Roles

One person wears all three hats; the value is in naming them so that time spent is deliberate.

Table 10: Role allocation across the project.

Role	Responsibility	Share of effort
Platform	Minikube cluster, manifests, Services, probes, resource policy, PV/PVC, teardown and reprovision correctness.	~45%
Services	The four FastAPI applications, the metrics sidecar, Dockerfiles, the ONNX integration and the mock classifier.	~35%
Documentation and demo	This report, the diagrams, the final report, evidence capture, and the demonstration video.	~20%

The Platform share is the largest on purpose: the project is graded on Kubernetes orchestration, not on application code, and the mock classifier exists specifically so that Services work can never become the critical path (§6.1, R8).

7.2 Cadence

Table 11: Working cadence. Self-imposed, since no teammate will notice a missed checkpoint.

Checkpoint	Frequency	Output
Working session	2–3 per week, ~2 hours	Commits on a feature branch; nothing left uncommitted at session end.
Self-review	Weekly, end of week	Re-read the milestone requirements against what exists; update the risk register; write down what is now blocked.
Milestone gate	Per milestone	Full deliverable checklist walked item by item before submission, against the traceability table in this document.

Because no one else will notice a missed checkpoint, the weekly self-review is written down in the repository rather than done mentally — a short dated note per week in `docs/journal.md`. That note is also the raw material for the "individual contribution" section the brief asks the final report to include.

7.3 Definition of Done

A unit of work is done only when **all** of the following hold. Partial completion is tracked as not done, not as mostly done.

1. **It applies from nothing.** The relevant manifests apply cleanly onto a freshly started cluster with no manual steps and no pre-existing state.
2. **It answers.** The endpoint or behaviour responds correctly to a `curl` command that is committed to the repository, not typed from memory.
3. **It is tested.** The corresponding row of the Table 9 matrix passes, and the command and its output are captured.
4. **It survives teardown.** `kubectl delete -f . && kubectl apply -f .` still works afterwards.
5. **It is documented.** Any decision that diverges from this design document is recorded, with the reason, in the same commit that makes the change.
6. **It is committed.** Work exists on a branch with a message explaining *why*, and the working tree is clean.

Criterion 5 matters most to the project as a whole. This document will be wrong about something — designs written before implementation always are — and the value of the final report depends on those divergences being recorded as they happen rather than reconstructed from memory at the end.

7.4 Milestone schedule

Table 12: Milestone schedule. Dates are filled in from the course calendar.

Milestone	Scope	Target
M1	Architecture and design (this document)	[M1 due date]
M2	Four Deployments, four Services, sidecar, scaling and self-healing evidence; no persistence	[M2 due date]
M3	PV/PVC, persistence verification, probes, resource limits, demo video, final report	[M3 due date]

Within M2 the order is deliberate: manifests with the mock classifier first, so that a working end-to-end pipeline exists as early as possible, then the real model, then the sidecar, then the scaling and self-healing evidence. The riskiest work (R1, R3 — image size and resource contention) is scheduled first, while there is still time to change the approach.